

Section 9.4 - Model interpretation

December 3, 2025

1 Interpreting what ConvNets learn

1.1 Visualizing intermediate activations

- For ConvNets, feature maps represents visual concepts
- What - Visualize output of layers
- Why - Understand how an input is decomposed into feature maps
- How - Use a model that returns layer activations
- Sparsity: the pattern encoded by that filter is not present in the input
- DL network acts as an information distillation pipeline: irrelevant infor is filtered out while useful infor is magnified & refined.

1.1.1 Load the saved convnet model

```
[1]: # Hide the warning messages
import tensorflow as tf
import logging

tf.get_logger().setLevel(logging.ERROR)

from tensorflow import keras

model = keras.models.load_model(
    "convnet_from_scratch_with_augmentation.h5",
    compile=False
)

model.compile(
    optimizer=keras.optimizers.Adam(),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

Model: "model"

Layer (type)	Output Shape	Param #
--------------	--------------	---------

input_1 (InputLayer)	[(None, 180, 180, 3)]	0
sequential (Sequential)	(None, 180, 180, 3)	0
rescaling (Rescaling)	(None, 180, 180, 3)	0
conv2d (Conv2D)	(None, 178, 178, 32)	896
max_pooling2d (MaxPooling2D)	(None, 89, 89, 32)	0
conv2d_1 (Conv2D)	(None, 87, 87, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 43, 43, 64)	0
conv2d_2 (Conv2D)	(None, 41, 41, 128)	73856
max_pooling2d_2 (MaxPooling2D)	(None, 20, 20, 128)	0
conv2d_3 (Conv2D)	(None, 18, 18, 256)	295168
max_pooling2d_3 (MaxPooling2D)	(None, 9, 9, 256)	0
conv2d_4 (Conv2D)	(None, 7, 7, 256)	590080
flatten (Flatten)	(None, 12544)	0
dropout (Dropout)	(None, 12544)	0
dense (Dense)	(None, 1)	12545

```
=====
Total params: 991,041
Trainable params: 991,041
Non-trainable params: 0
-----
```

1.1.2 Preprocessing a single image

```
[2]: from tensorflow import keras
import numpy as np

img_path = keras.utils.get_file(
    fname="cat.jpg",
```

```

origin="https://img-datasets.s3.amazonaws.com/cat.jpg")

def get_img_array(img_path, target_size):
    img = keras.utils.load_img(img_path, target_size=target_size)
    array = keras.utils.img_to_array(img)
    array = np.expand_dims(array, axis=0)
    return array

img_tensor = get_img_array(img_path, target_size=(180, 180))

```

1.1.3 Displaying the test picture

```

[3]: import matplotlib.pyplot as plt

plt.axis("off")
plt.imshow(img_tensor[0].astype("uint8"))
plt.show()

```



1.1.4 A model returning layer activations

```

[4]: from tensorflow.keras import layers

layer_outputs = []
layer_names = []
for layer in model.layers:
    if isinstance(layer, (layers.Conv2D, layers.MaxPooling2D)):

```

```
        layer_outputs.append(layer.output)
        layer_names.append(layer.name)
activation_model = keras.Model(inputs=model.input, outputs=layer_outputs)
```

1.1.5 Computing layer activations

```
[5]: activations = activation_model.predict(img_tensor)
```

```
1/1 [=====] - 0s 390ms/step
```

```
[6]: for k in range(len(activations)):
      print(activations[k].shape)
```

```
(1, 178, 178, 32)
(1, 89, 89, 32)
(1, 87, 87, 64)
(1, 43, 43, 64)
(1, 41, 41, 128)
(1, 20, 20, 128)
(1, 18, 18, 256)
(1, 9, 9, 256)
(1, 7, 7, 256)
```

```
[7]: first_layer_activation = activations[0]
      print(first_layer_activation.shape)
```

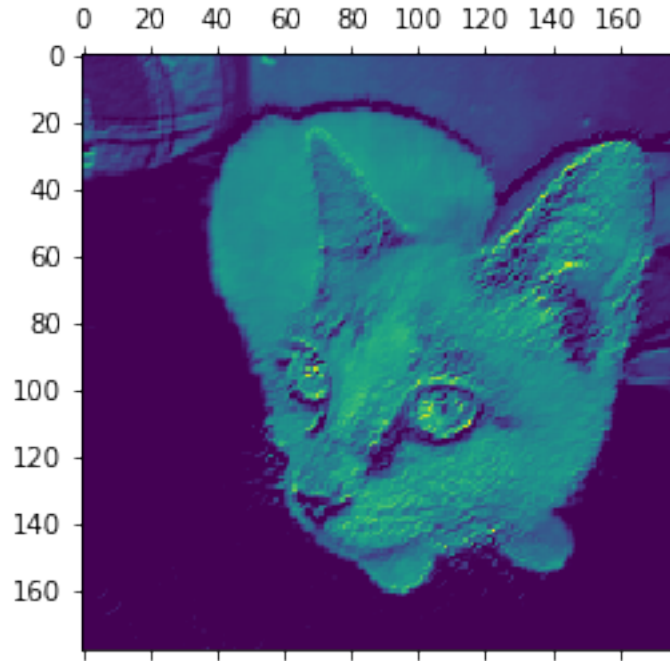
```
(1, 178, 178, 32)
```

1.1.6 Visualizing the fourth channel

```
[8]: import matplotlib.pyplot as plt
```

```
plt.matshow(first_layer_activation[0, :, :, 3], cmap="viridis")
```

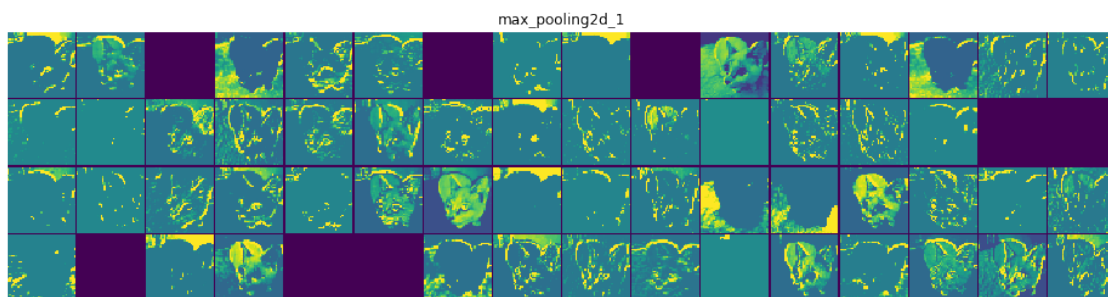
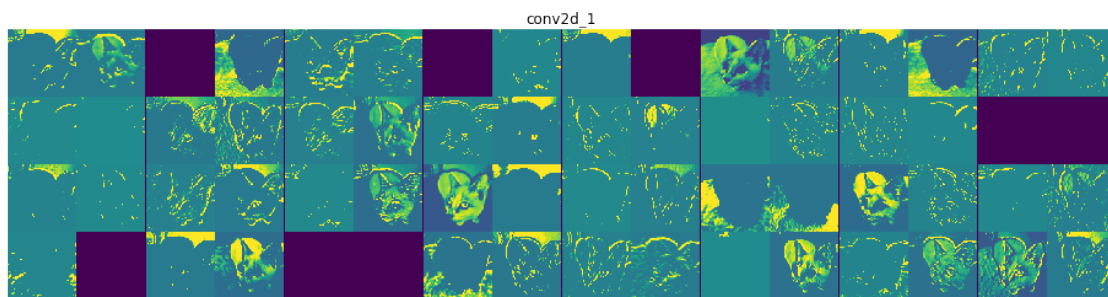
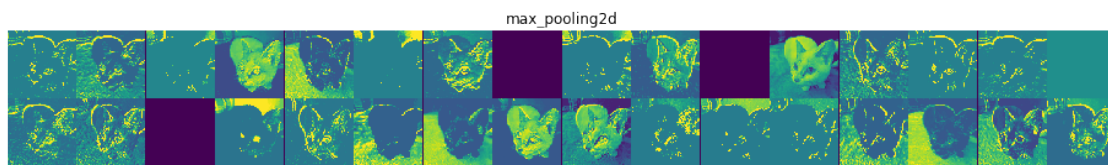
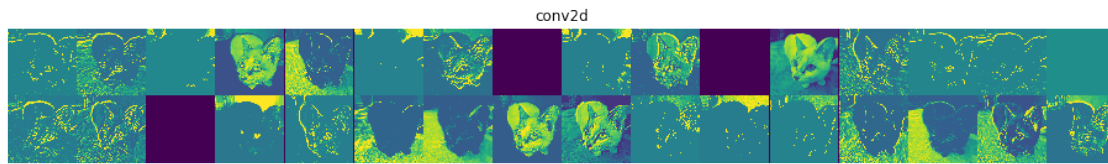
```
[8]: <matplotlib.image.AxesImage at 0x7f53c808d6d0>
```



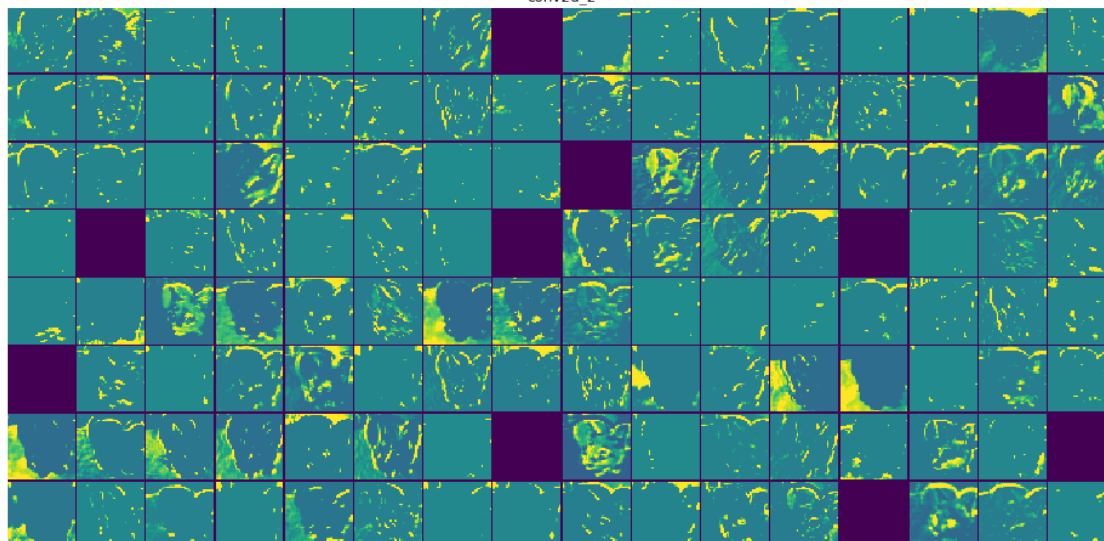
1.1.7 Visualizing every intermediate activation

```
[9]: images_per_row = 16
for layer_name, layer_activation in zip(layer_names, activations):
    n_features = layer_activation.shape[-1]
    size = layer_activation.shape[1]
    n_cols = n_features // images_per_row
    display_grid = np.zeros(((size + 1) * n_cols - 1,
                             images_per_row * (size + 1) - 1))
    for col in range(n_cols):
        for row in range(images_per_row):
            channel_index = col * images_per_row + row
            channel_image = layer_activation[0, :, :, channel_index].copy()
            if channel_image.sum() != 0:
                channel_image -= channel_image.mean()
                channel_image /= channel_image.std()
                channel_image *= 64
                channel_image += 128
            channel_image = np.clip(channel_image, 0, 255).astype("uint8")
            display_grid[
                col * (size + 1): (col + 1) * size + col,
                row * (size + 1) : (row + 1) * size + row] = channel_image
    scale = 1. / size
    plt.figure(figsize=(scale * display_grid.shape[1],
                        scale * display_grid.shape[0]))
```

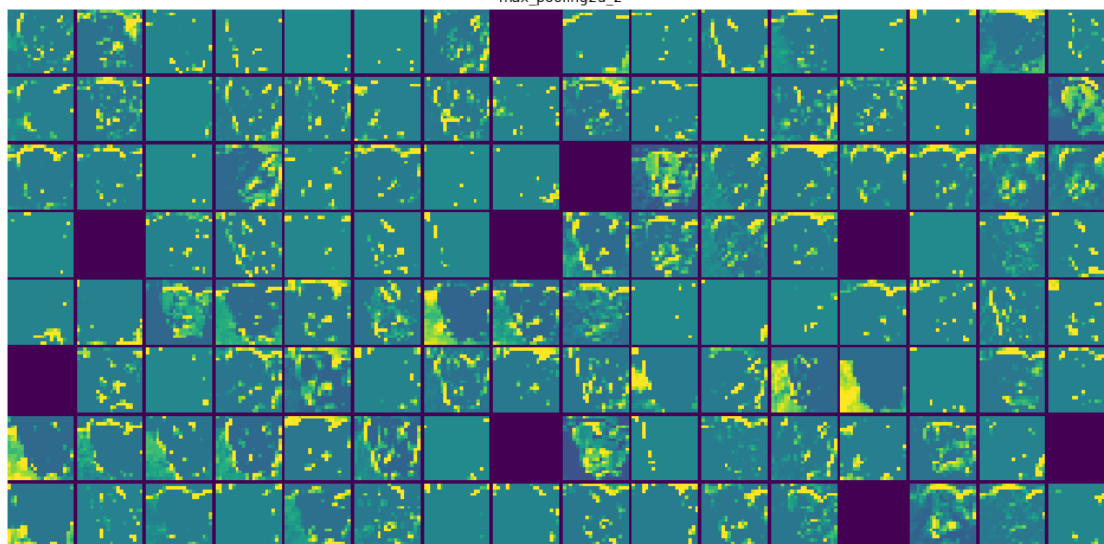
```
plt.title(layer_name)
plt.grid(False)
plt.axis("off")
plt.imshow(display_grid, aspect="auto", cmap="viridis")
```

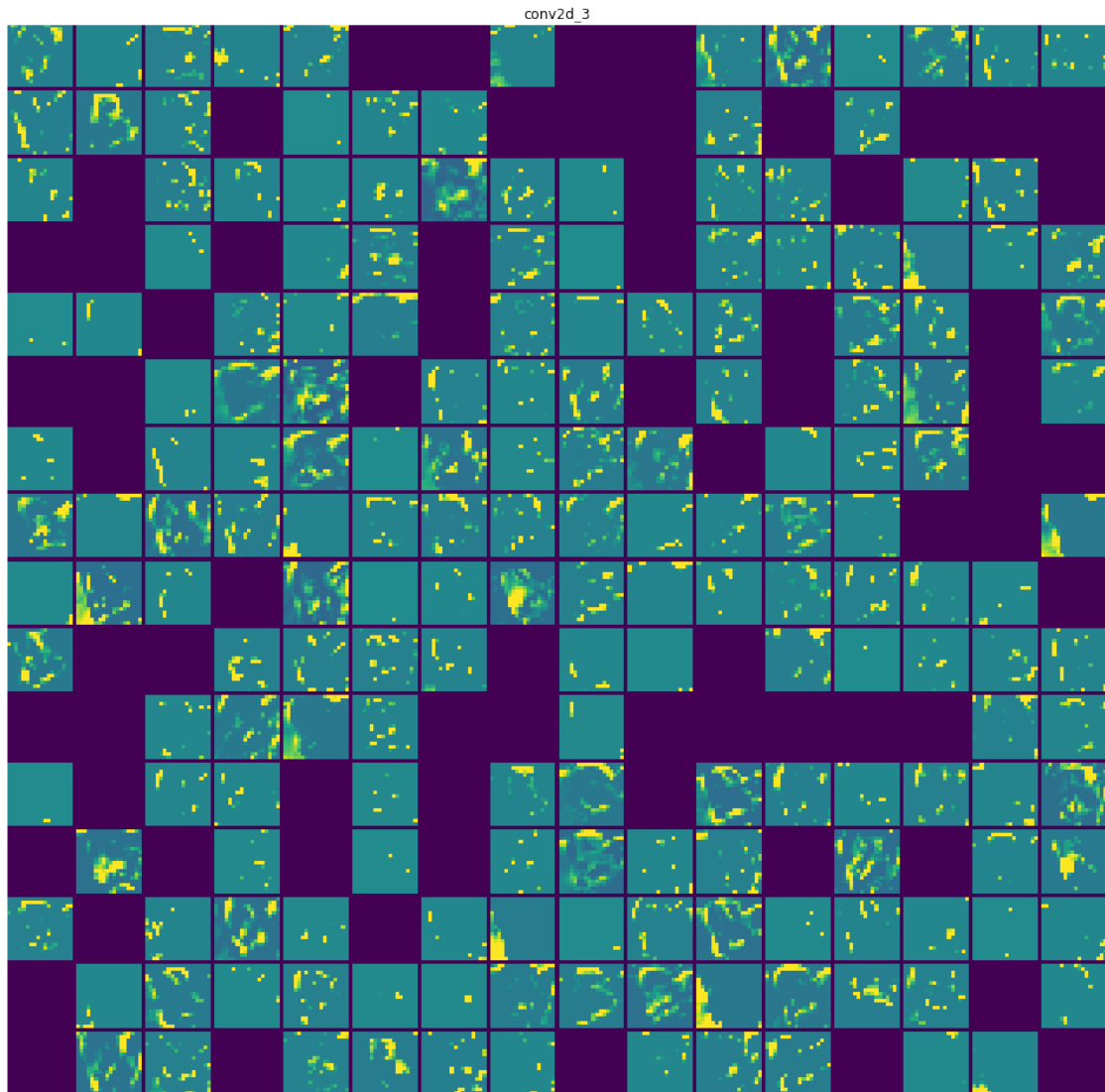


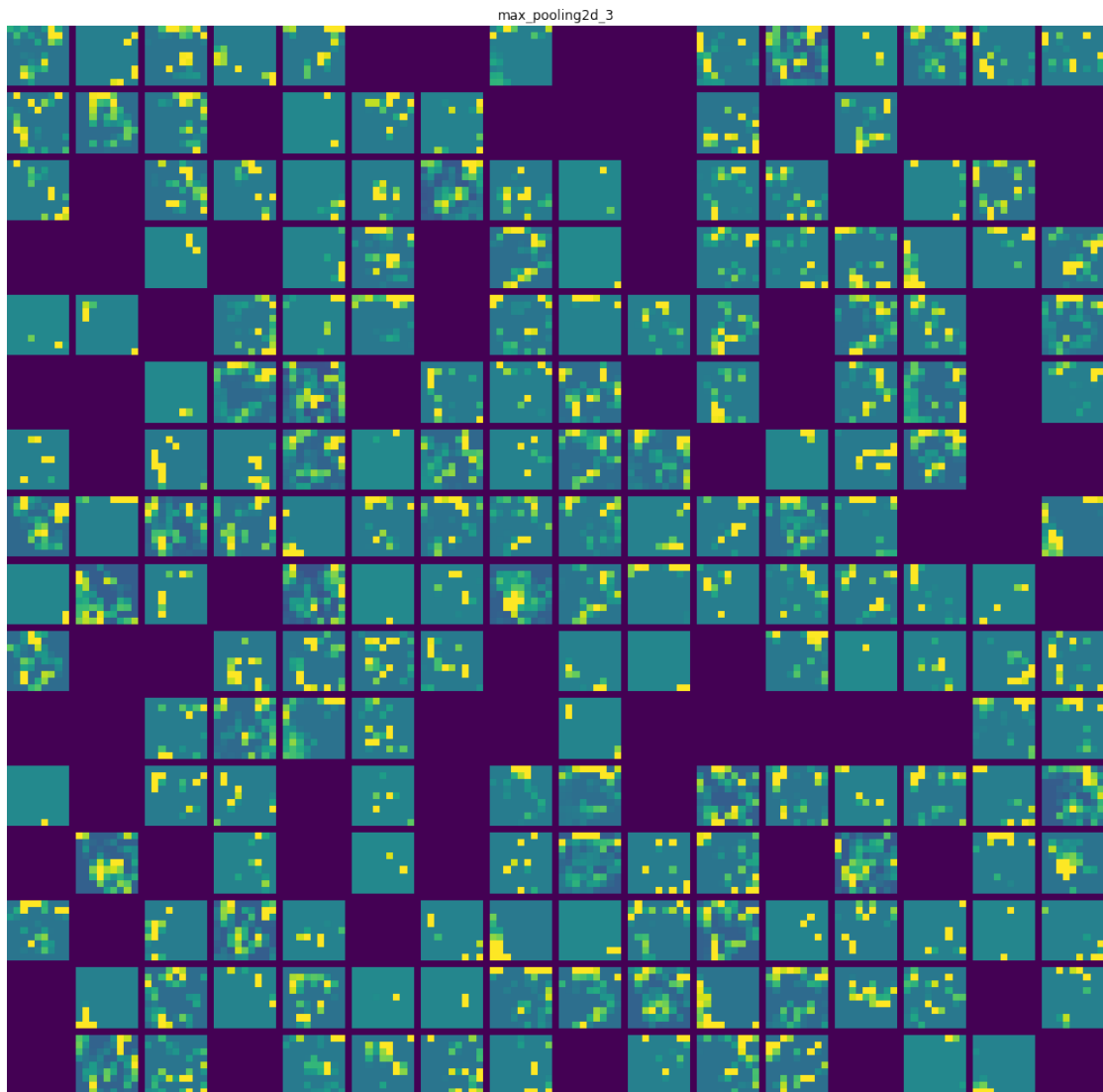
conv2d_2

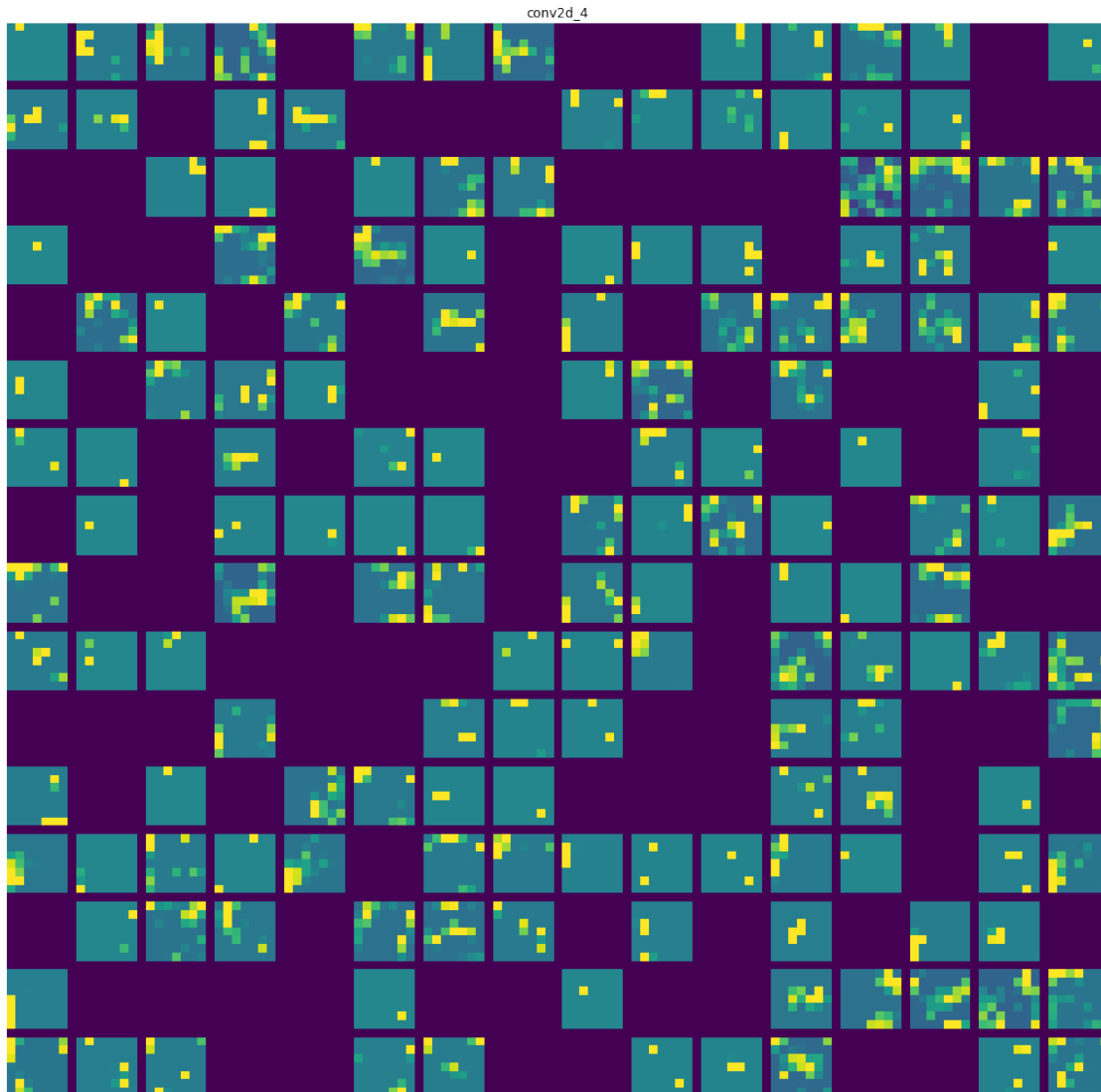


max_pooling2d_2









1.2 Visualizing convnet filters

- Apply GD to maximize the response of a filter
- Display the pattern a filter is meant to respond
- Alternative: directly look at the filters
- Feature extractor: return output of a layer
- Cost function: how much an input activates a filter
- Use GradientTape object
- Use predict() as just output value
- Given layer, filter index -> Return pattern maximizing the activation of the filter
- Deeper layers -> more complex & refined filters

1.2.1 Xception convolutional base

```
[10]: from tensorflow import keras

model = keras.applications.xception.Xception(
    weights="imagenet",
    include_top=False)

model.summary()
```

Model: "xception"

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, None, None, 0 3)]	0	[]
block1_conv1 (Conv2D) ['input_1[0][0]']	(None, None, None, 864 32)	864	
block1_conv1_bn (BatchNormaliz ['block1_conv1[0][0]'] ation)	(None, None, None, 128 32)	128	
block1_conv1_act (Activation) ['block1_conv1_bn[0][0]']	(None, None, None, 0 32)	0	
block1_conv2 (Conv2D) ['block1_conv1_act[0][0]']	(None, None, None, 18432 64)	18432	
block1_conv2_bn (BatchNormaliz ['block1_conv2[0][0]'] ation)	(None, None, None, 256 64)	256	
block1_conv2_act (Activation) ['block1_conv2_bn[0][0]']	(None, None, None, 0 64)	0	
block2_sepconv1 (SeparableConv ['block1_conv2_act[0][0]'] 2D)	(None, None, None, 8768 128)	8768	
block2_sepconv1_bn (BatchNorma	(None, None, None, 512	512	

['block2_sepconv1[0][0]'] lization)	(None, None, None, 128)	
block2_sepconv2_act (Activatio ['block2_sepconv1_bn[0][0]'] n)	(None, None, None, 128)	0
block2_sepconv2 (SeparableConv ['block2_sepconv2_act[0][0]'] 2D)	(None, None, None, 128)	17536
block2_sepconv2_bn (BatchNorma ['block2_sepconv2[0][0]'] lization)	(None, None, None, 128)	512
conv2d (Conv2D) ['block1_conv2_act[0][0]']	(None, None, None, 128)	8192
block2_pool (MaxPooling2D) ['block2_sepconv2_bn[0][0]']	(None, None, None, 128)	0
batch_normalization (BatchNorm ['conv2d[0][0]'] alization)	(None, None, None, 128)	512
add (Add) ['block2_pool[0][0]', 'batch_normalization[0][0]']	(None, None, None, 128)	0
block3_sepconv1_act (Activatio n)	(None, None, None, 128)	0
block3_sepconv1 (SeparableConv ['block3_sepconv1_act[0][0]'] 2D)	(None, None, None, 256)	33920
block3_sepconv1_bn (BatchNorma ['block3_sepconv1[0][0]'] lization)	(None, None, None, 256)	1024
block3_sepconv2_act (Activatio ['block3_sepconv1_bn[0][0]'] n)	(None, None, None, 256)	0
block3_sepconv2 (SeparableConv	(None, None, None,	67840

['block3_sepconv2_act[0][0]'] 2D)	(None, None, None, 256)		
block3_sepconv2_bn (BatchNormaliza- tion)	(None, None, None, 256)	1024	
conv2d_1 (Conv2D)	(None, None, None, 256)	32768	['add[0][0]']
block3_pool (MaxPooling2D)	(None, None, None, 256)	0	
['block3_sepconv2_bn[0][0]']			
batch_normalization_1 (BatchNormaliza- tion)	(None, None, None, 256)	1024	
['conv2d_1[0][0]']			
add_1 (Add)	(None, None, None, 256)	0	
['block3_pool[0][0]', 'batch_normalization_1[0][0]']			
block4_sepconv1_act (Activation)	(None, None, None, 256)	0	['add_1[0][0]']
block4_sepconv1 (SeparableConv2D)	(None, None, None, 728)	188672	
['block4_sepconv1_act[0][0]']			
block4_sepconv1_bn (BatchNormalization)	(None, None, None, 728)	2912	
['block4_sepconv1[0][0]']			
block4_sepconv2_act (Activation)	(None, None, None, 728)	0	
['block4_sepconv1_bn[0][0]']			
block4_sepconv2 (SeparableConv2D)	(None, None, None, 728)	536536	
['block4_sepconv2_act[0][0]']			
block4_sepconv2_bn (BatchNormalization)	(None, None, None, 728)	2912	
['block4_sepconv2[0][0]']			
conv2d_2 (Conv2D)	(None, None, None, 186368)		['add_1[0][0]']

	728)		
block4_pool (MaxPooling2D)	(None, None, None,	0	
['block4_sepconv2_bn[0][0]']			
	728)		
batch_normalization_2 (BatchNo	(None, None, None,	2912	
['conv2d_2[0][0]']			
rmalization)	728)		
add_2 (Add)	(None, None, None,	0	
['block4_pool[0][0]',			
	728)		
'batch_normalization_2[0][0]']			
block5_sepconv1_act (Activatio	(None, None, None,	0	['add_2[0][0]']
n)	728)		
block5_sepconv1 (SeparableConv	(None, None, None,	536536	
['block5_sepconv1_act[0][0]']			
2D)	728)		
block5_sepconv1_bn (BatchNorma	(None, None, None,	2912	
['block5_sepconv1[0][0]']			
lization)	728)		
block5_sepconv2_act (Activatio	(None, None, None,	0	
['block5_sepconv1_bn[0][0]']			
n)	728)		
block5_sepconv2 (SeparableConv	(None, None, None,	536536	
['block5_sepconv2_act[0][0]']			
2D)	728)		
block5_sepconv2_bn (BatchNorma	(None, None, None,	2912	
['block5_sepconv2[0][0]']			
lization)	728)		
block5_sepconv3_act (Activatio	(None, None, None,	0	
['block5_sepconv2_bn[0][0]']			
n)	728)		
block5_sepconv3 (SeparableConv	(None, None, None,	536536	
['block5_sepconv3_act[0][0]']			
2D)	728)		
block5_sepconv3_bn (BatchNorma	(None, None, None,	2912	
['block5_sepconv3[0][0]']			

lization)	728)	
add_3 (Add)	(None, None, None,	0
['block5_sepconv3_bn[0][0]',	728)	'add_2[0][0]'
block6_sepconv1_act (Activatio	(None, None, None,	0
n)	728)	['add_3[0][0]']
block6_sepconv1 (SeparableConv	(None, None, None,	536536
['block6_sepconv1_act[0][0]']	728)	
2D)		
block6_sepconv1_bn (BatchNorma	(None, None, None,	2912
['block6_sepconv1[0][0]']	728)	
lization)		
block6_sepconv2_act (Activatio	(None, None, None,	0
['block6_sepconv1_bn[0][0]']	728)	
n)		
block6_sepconv2 (SeparableConv	(None, None, None,	536536
['block6_sepconv2_act[0][0]']	728)	
2D)		
block6_sepconv2_bn (BatchNorma	(None, None, None,	2912
['block6_sepconv2[0][0]']	728)	
lization)		
block6_sepconv3_act (Activatio	(None, None, None,	0
['block6_sepconv2_bn[0][0]']	728)	
n)		
block6_sepconv3 (SeparableConv	(None, None, None,	536536
['block6_sepconv3_act[0][0]']	728)	
2D)		
block6_sepconv3_bn (BatchNorma	(None, None, None,	2912
['block6_sepconv3[0][0]']	728)	
lization)		
add_4 (Add)	(None, None, None,	0
['block6_sepconv3_bn[0][0]',	728)	'add_3[0][0]'
block7_sepconv1_act (Activatio	(None, None, None,	0
n)	728)	['add_4[0][0]']

block7_sepconv1 (SeparableConv ['block7_sepconv1_act[0][0]'] 2D)	(None, None, None, 728)	536536	
block7_sepconv1_bn (BatchNorma ['block7_sepconv1[0][0]'] lization)	(None, None, None, 728)	2912	
block7_sepconv2_act (Activatio ['block7_sepconv1_bn[0][0]'] n)	(None, None, None, 728)	0	
block7_sepconv2 (SeparableConv ['block7_sepconv2_act[0][0]'] 2D)	(None, None, None, 728)	536536	
block7_sepconv2_bn (BatchNorma ['block7_sepconv2[0][0]'] lization)	(None, None, None, 728)	2912	
block7_sepconv3_act (Activatio ['block7_sepconv2_bn[0][0]'] n)	(None, None, None, 728)	0	
block7_sepconv3 (SeparableConv ['block7_sepconv3_act[0][0]'] 2D)	(None, None, None, 728)	536536	
block7_sepconv3_bn (BatchNorma ['block7_sepconv3[0][0]'] lization)	(None, None, None, 728)	2912	
add_5 (Add) ['block7_sepconv3_bn[0][0]',	(None, None, None, 728)	0	'add_4[0][0]']
block8_sepconv1_act (Activatio n)	(None, None, None, 728)	0	['add_5[0][0]']
block8_sepconv1 (SeparableConv ['block8_sepconv1_act[0][0]'] 2D)	(None, None, None, 728)	536536	
block8_sepconv1_bn (BatchNorma ['block8_sepconv1[0][0]'] lization)	(None, None, None, 728)	2912	

block8_sepconv2_act (Activation)	(None, None, None, 0)	
['block8_sepconv1_bn[0][0]']	728)	
block8_sepconv2 (SeparableConv2D)	(None, None, None, 536536)	
['block8_sepconv2_act[0][0]']	728)	
block8_sepconv2_bn (BatchNormalization)	(None, None, None, 2912)	
['block8_sepconv2[0][0]']	728)	
block8_sepconv3_act (Activation)	(None, None, None, 0)	
['block8_sepconv2_bn[0][0]']	728)	
block8_sepconv3 (SeparableConv2D)	(None, None, None, 536536)	
['block8_sepconv3_act[0][0]']	728)	
block8_sepconv3_bn (BatchNormalization)	(None, None, None, 2912)	
['block8_sepconv3[0][0]']	728)	
add_6 (Add)	(None, None, None, 0)	
['block8_sepconv3_bn[0][0]',	728)	'add_5[0][0]']
block9_sepconv1_act (Activation)	(None, None, None, 0)	['add_6[0][0]']
['block9_sepconv1_bn[0][0]']	728)	
block9_sepconv1 (SeparableConv2D)	(None, None, None, 536536)	
['block9_sepconv1_act[0][0]']	728)	
block9_sepconv1_bn (BatchNormalization)	(None, None, None, 2912)	
['block9_sepconv1[0][0]']	728)	
block9_sepconv2_act (Activation)	(None, None, None, 0)	
['block9_sepconv1_bn[0][0]']	728)	
block9_sepconv2 (SeparableConv2D)	(None, None, None, 536536)	
['block9_sepconv2_act[0][0]']	728)	
block9_sepconv2_bn (BatchNormalization)	(None, None, None, 2912)	

['block9_sepconv2[0][0]'] lization)	(None, None, None, 0 728)	
block9_sepconv3_act (Activatio ['block9_sepconv2_bn[0][0]'] n)	(None, None, None, 0 728)	
block9_sepconv3 (SeparableConv ['block9_sepconv3_act[0][0]'] 2D)	(None, None, None, 536536 728)	
block9_sepconv3_bn (BatchNorma ['block9_sepconv3[0][0]'] lization)	(None, None, None, 2912 728)	
add_7 (Add) ['block9_sepconv3_bn[0][0]', 728)	(None, None, None, 0 728)	'add_6[0][0]']
block10_sepconv1_act (Activati on)	(None, None, None, 0 728)	['add_7[0][0]']
block10_sepconv1 (SeparableCon ['block10_sepconv1_act[0][0]'] v2D)	(None, None, None, 536536 728)	
block10_sepconv1_bn (BatchNorm ['block10_sepconv1[0][0]'] alization)	(None, None, None, 2912 728)	
block10_sepconv2_act (Activati ['block10_sepconv1_bn[0][0]'] on)	(None, None, None, 0 728)	
block10_sepconv2 (SeparableCon ['block10_sepconv2_act[0][0]'] v2D)	(None, None, None, 536536 728)	
block10_sepconv2_bn (BatchNorm ['block10_sepconv2[0][0]'] alization)	(None, None, None, 2912 728)	
block10_sepconv3_act (Activati ['block10_sepconv2_bn[0][0]'] on)	(None, None, None, 0 728)	
block10_sepconv3 (SeparableCon (None, None, None, 536536		

```

['block10_sepconv3_act[0][0]']
v2D)                                728)

block10_sepconv3_bn (BatchNorm (None, None, None, 2912
['block10_sepconv3[0][0]']
alization)                                728)

add_8 (Add)                            (None, None, None, 0
['block10_sepconv3_bn[0][0]',
728)                                'add_7[0][0]']

block11_sepconv1_act (Activati (None, None, None, 0
on)                                728)    ['add_8[0][0]']

block11_sepconv1 (SeparableCon (None, None, None, 536536
['block11_sepconv1_act[0][0]']
v2D)                                728)

block11_sepconv1_bn (BatchNorm (None, None, None, 2912
['block11_sepconv1[0][0]']
alization)                                728)

block11_sepconv2_act (Activati (None, None, None, 0
['block11_sepconv1_bn[0][0]']
on)                                728)

block11_sepconv2 (SeparableCon (None, None, None, 536536
['block11_sepconv2_act[0][0]']
v2D)                                728)

block11_sepconv2_bn (BatchNorm (None, None, None, 2912
['block11_sepconv2[0][0]']
alization)                                728)

block11_sepconv3_act (Activati (None, None, None, 0
['block11_sepconv2_bn[0][0]']
on)                                728)

block11_sepconv3 (SeparableCon (None, None, None, 536536
['block11_sepconv3_act[0][0]']
v2D)                                728)

block11_sepconv3_bn (BatchNorm (None, None, None, 2912
['block11_sepconv3[0][0]']
alization)                                728)

add_9 (Add)                            (None, None, None, 0
['block11_sepconv3_bn[0][0]',

```

	728)		'add_8[0][0]'
block12_sepconv1_act (Activation)	(None, None, None, 728)	0	['add_9[0][0]']
block12_sepconv1 (SeparableConv2D)	(None, None, None, 728)	536536	
block12_sepconv1_bn (BatchNormalization)	(None, None, None, 728)	2912	
block12_sepconv2_act (Activation)	(None, None, None, 728)	0	
block12_sepconv2 (SeparableConv2D)	(None, None, None, 728)	536536	
block12_sepconv2_bn (BatchNormalization)	(None, None, None, 728)	2912	
block12_sepconv3_act (Activation)	(None, None, None, 728)	0	
block12_sepconv3 (SeparableConv2D)	(None, None, None, 728)	536536	
block12_sepconv3_bn (BatchNormalization)	(None, None, None, 728)	2912	
add_10 (Add)	(None, None, None, 728)	0	'add_9[0][0]'
block13_sepconv1_act (Activation)	(None, None, None, 728)	0	
block13_sepconv1 (SeparableConv2D)	(None, None, None, 728)	536536	

v2D)	728)	
block13_sepconv1_bn (BatchNorm ['block13_sepconv1[0][0]'] alization)	(None, None, None, 728)	2912
block13_sepconv2_act (Activati ['block13_sepconv1_bn[0][0]'] on)	(None, None, None, 728)	0
block13_sepconv2 (SeparableCon ['block13_sepconv2_act[0][0]'] v2D)	(None, None, None, 1024)	752024
block13_sepconv2_bn (BatchNorm ['block13_sepconv2[0][0]'] alization)	(None, None, None, 1024)	4096
conv2d_3 (Conv2D) ['add_10[0][0]']	(None, None, None, 1024)	745472
block13_pool (MaxPooling2D) ['block13_sepconv2_bn[0][0]']	(None, None, None, 1024)	0
batch_normalization_3 (BatchNo ['conv2d_3[0][0]'] rmalization)	(None, None, None, 1024)	4096
add_11 (Add) ['block13_pool[0][0]', 'batch_normalization_3[0][0]']	(None, None, None, 1024)	0
block14_sepconv1 (SeparableCon ['add_11[0][0]'] v2D)	(None, None, None, 1536)	1582080
block14_sepconv1_bn (BatchNorm ['block14_sepconv1[0][0]'] alization)	(None, None, None, 1536)	6144
block14_sepconv1_act (Activati ['block14_sepconv1_bn[0][0]'] on)	(None, None, None, 1536)	0
block14_sepconv2 (SeparableCon	(None, None, None,	3159552

```
['block14_sepconv1_act[0][0]']
v2D)                                2048)

block14_sepconv2_bn (BatchNorm (None, None, None, 8192
['block14_sepconv2[0][0]']
alization)                        2048)

block14_sepconv2_act (Activati (None, None, None, 0
['block14_sepconv2_bn[0][0]']
on)                                2048)
```

```
=====
=====
Total params: 20,861,480
Trainable params: 20,806,952
Non-trainable params: 54,528
-----
-----
```

1.2.2 Layer names

```
[11]: for layer in model.layers:
        if isinstance(layer, (keras.layers.Conv2D, keras.layers.SeparableConv2D)):
            print(layer.name)
```

```
block1_conv1
block1_conv2
block2_sepconv1
block2_sepconv2
conv2d
block3_sepconv1
block3_sepconv2
conv2d_1
block4_sepconv1
block4_sepconv2
conv2d_2
block5_sepconv1
block5_sepconv2
block5_sepconv3
block6_sepconv1
block6_sepconv2
block6_sepconv3
block7_sepconv1
block7_sepconv2
block7_sepconv3
block8_sepconv1
block8_sepconv2
block8_sepconv3
```

```

block9_sepconv1
block9_sepconv2
block9_sepconv3
block10_sepconv1
block10_sepconv2
block10_sepconv3
block11_sepconv1
block11_sepconv2
block11_sepconv3
block12_sepconv1
block12_sepconv2
block12_sepconv3
block13_sepconv1
block13_sepconv2
conv2d_3
block14_sepconv1
block14_sepconv2

```

1.2.3 A feature extractor

```

[12]: layer_name = "block3_sepconv1"
      layer = model.get_layer(name=layer_name)
      feature_extractor = keras.Model(inputs=model.input, outputs=layer.output)

      feature_extractor.summary()

```

Model: "model_1"

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, None, None, 0 3)]	0	[]
block1_conv1 (Conv2D) ['input_1[0][0]']	(None, None, None, 32)	864	
block1_conv1_bn (BatchNormaliz ['block1_conv1[0][0]'] ation)	(None, None, None, 32)	128	
block1_conv1_act (Activation) ['block1_conv1_bn[0][0]']	(None, None, None, 32)	0	
block1_conv2 (Conv2D)	(None, None, None, 18432)		

```

['block1_conv1_act[0][0]']
64)

block1_conv2_bn (BatchNormaliz (None, None, None, 256
['block1_conv2[0][0]']
ation)
64)

block1_conv2_act (Activation) (None, None, None, 0
['block1_conv2_bn[0][0]']
64)

block2_sepconv1 (SeparableConv (None, None, None, 8768
['block1_conv2_act[0][0]']
2D)
128)

block2_sepconv1_bn (BatchNorma (None, None, None, 512
['block2_sepconv1[0][0]']
lization)
128)

block2_sepconv2_act (Activatio (None, None, None, 0
['block2_sepconv1_bn[0][0]']
n)
128)

block2_sepconv2 (SeparableConv (None, None, None, 17536
['block2_sepconv2_act[0][0]']
2D)
128)

block2_sepconv2_bn (BatchNorma (None, None, None, 512
['block2_sepconv2[0][0]']
lization)
128)

conv2d (Conv2D) (None, None, None, 8192
['block1_conv2_act[0][0]']
128)

block2_pool (MaxPooling2D) (None, None, None, 0
['block2_sepconv2_bn[0][0]']
128)

batch_normalization (BatchNorm (None, None, None, 512
['conv2d[0][0]']
alization)
128)

add (Add) (None, None, None, 0
['block2_pool[0][0]',
128)
'batch_normalization[0][0]']

```



```
block3_sepconv1_act (Activation (None, None, None, 0 ['add[0][0]']
n)                                128)
```

```
block3_sepconv1 (SeparableConv (None, None, None, 33920
['block3_sepconv1_act[0][0]']
2D)                                256)
```

```
=====
=====
```

```
Total params: 89,632
Trainable params: 88,672
Non-trainable params: 960
```

```
-----
-----
```

1.2.4 Using the feature extractor

```
[13]: activation = feature_extractor(
        keras.applications.xception.preprocess_input(img_tensor)
    )

    print(activation.shape)
```

```
(1, 44, 44, 256)
```

1.2.5 Computing the value

```
[14]: import tensorflow as tf

def compute_loss(image, filter_index):
    activation = feature_extractor(image)
    filter_activation = activation[:, 2:-2, 2:-2, filter_index]
    return tf.reduce_mean(filter_activation)
```

1.2.6 Value maximization

```
[15]: @tf.function
def gradient_ascent_step(image, filter_index, learning_rate):
    with tf.GradientTape() as tape:
        tape.watch(image)
        loss = compute_loss(image, filter_index)
    grads = tape.gradient(loss, image)
    grads = tf.math.l2_normalize(grads)
    image += learning_rate * grads
    return image
```

1.2.7 Function to generate filter visualizations

```
[16]: img_width = 200
      img_height = 200

      def generate_filter_pattern(filter_index):
          iterations = 30
          learning_rate = 10.
          image = tf.random.uniform(
              minval=0.4,
              maxval=0.6,
              shape=(1, img_width, img_height, 3))
          for i in range(iterations):
              image = gradient_ascent_step(image, filter_index, learning_rate)
          return image[0].numpy()
```

1.2.8 Tensor into image

```
[17]: def deprocess_image(image):
      image -= image.mean()
      image /= image.std()
      image *= 64
      image += 128
      image = np.clip(image, 0, 255).astype("uint8")
      image = image[25:-25, 25:-25, :]
      return image
```

```
[18]: import matplotlib.pyplot as plt

      plt.axis("off")
      plt.imshow(deprocess_image(generate_filter_pattern(filter_index=6)))
```

```
[18]: <matplotlib.image.AxesImage at 0x7f53c8220c40>
```



1.2.9 All filter response patterns in a layer

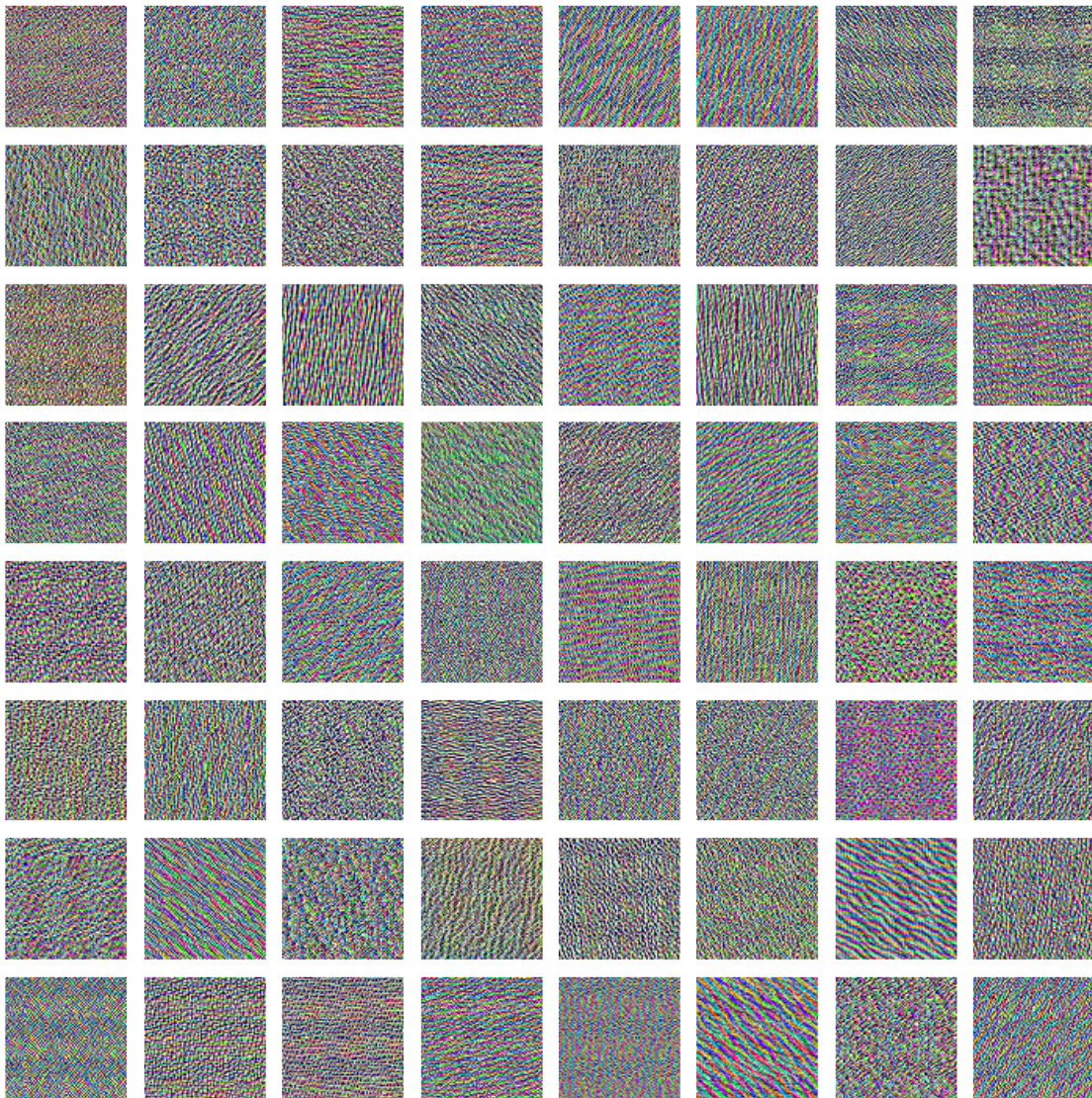
```
[19]: all_images = []
n_images = 64
for filter_index in range(n_images):
    # print(f"Processing filter {filter_index}")
    image = deprocess_image(
        generate_filter_pattern(filter_index)
    )
    all_images.append(image)

margin = 5
images_per_row = 8
cropped_width = img_width - 25 * 2
cropped_height = img_height - 25 * 2
width = images_per_row * cropped_width + (images_per_row - 1) * margin
height = images_per_row * cropped_height + (images_per_row - 1) * margin
stitched_filters = np.zeros((width, height, 3))

for i in range(images_per_row):
    for j in range(images_per_row):
        image = all_images[i * images_per_row + j]
        stitched_filters[
            (cropped_width + margin) * i : (cropped_width + margin) * i + ↵
            ↵cropped_width,
            (cropped_height + margin) * j : (cropped_height + margin) * j
            + cropped_height,
```

```
    :,  
    ] = image
```

```
[20]: import matplotlib.pyplot as plt  
  
n_cols = n_images // images_per_row  
  
fig, axes = plt.subplots(n_cols, images_per_row, figsize=(10, 10))  
  
for i, ax in enumerate(axes.flat): # Flatten axes for easy iteration  
    ax.imshow(all_images[i])  
    ax.axis("off")  
  
plt.tight_layout()  
plt.show()
```



1.3 Visualizing heatmaps of class activation

- Class activation map (CAM) visualization
- Produce heat maps of class activation over input image
- Understand decision process - model interpretability
- How important each location is to the class

- 1) Global averaging pooling (GAP) $\rightarrow$ C features (number of channels)
- 2) Each feature $k \rightarrow$ prediction weight w_k^c for class c
- 3) Model predicts class c (where is the evidence of this class?) $\rightarrow$ $CAM_c(i,j) = \sum_k w_k^c A_{ijk}$

1.3.1 The Xception network

```
[21]: model = keras.applications.xception.Xception(weights="imagenet")
```

1.3.2 Preprocessing an input image for Xception

```
[22]: img_path = keras.utils.get_file(
      fname="elephant.jpg",
      origin="https://img-datasets.s3.amazonaws.com/elephant.jpg")

def get_img_array(img_path, target_size):
    img = keras.utils.load_img(img_path, target_size=target_size)
    array = keras.utils.img_to_array(img)
    array = np.expand_dims(array, axis=0)
    array = keras.applications.xception.preprocess_input(array)
    return array

img_array = get_img_array(img_path, target_size=(299, 299))

preds = model.predict(img_array)
print(keras.applications.xception.decode_predictions(preds, top=3)[0])
print(np.argmax(preds[0]))
```

```
1/1 [=====] - 1s 875ms/step
[('n02504458', 'African_elephant', 0.86993945), ('n01871265', 'tusker',
0.076956354), ('n02504013', 'Indian_elephant', 0.023541817)]
386
```

1.3.3 Model returning the last convolutional output

```
[23]: last_conv_layer_name = "block14_sepconv2_act"
classifier_layer_names = [
    "avg_pool",
    "predictions",
]
last_conv_layer = model.get_layer(last_conv_layer_name)
last_conv_layer_model = keras.Model(model.inputs, last_conv_layer.output)
```

1.3.4 Reapply the classifier on top of the last convolutional output

```
[24]: classifier_input = keras.Input(shape=last_conv_layer.output.shape[1:])
x = classifier_input
for layer_name in classifier_layer_names:
    x = model.get_layer(layer_name)(x)
classifier_model = keras.Model(classifier_input, x)
```

1.3.5 Retrieving the gradients of the top predicted class

```
[25]: import tensorflow as tf

with tf.GradientTape() as tape:
    last_conv_layer_output = last_conv_layer_model(img_array)
    tape.watch(last_conv_layer_output)
    preds = classifier_model(last_conv_layer_output)
    top_pred_index = tf.argmax(preds[0])
    top_class_channel = preds[:, top_pred_index]

grads = tape.gradient(top_class_channel, last_conv_layer_output)
```

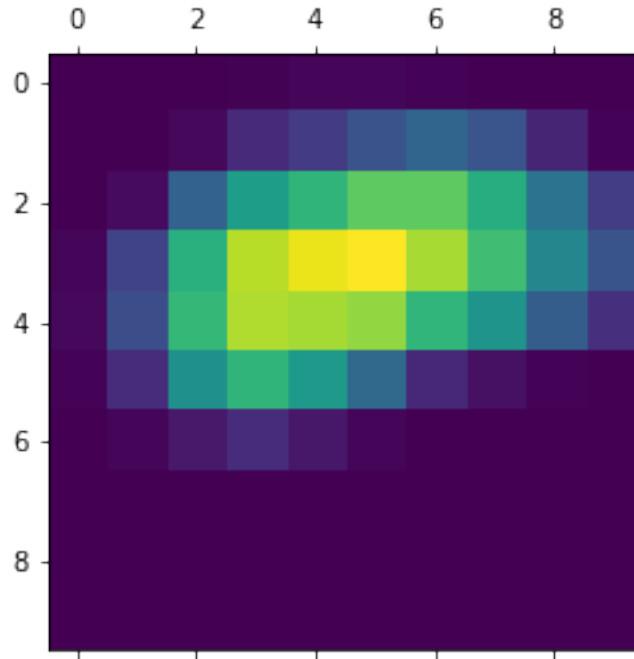
1.3.6 Gradient pooling and channel-importance weighting

```
[26]: pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2)).numpy()
last_conv_layer_output = last_conv_layer_output.numpy()[0]
for i in range(pooled_grads.shape[-1]):
    last_conv_layer_output[:, :, i] *= pooled_grads[i]
heatmap = np.mean(last_conv_layer_output, axis=-1)
```

1.3.7 Heatmap post-processing

```
[27]: heatmap = np.maximum(heatmap, 0)
heatmap /= np.max(heatmap)
plt.matshow(heatmap)
```

```
[27]: <matplotlib.image.AxesImage at 0x7f52e043cd90>
```



1.3.8 Superimposing on the original picture

```
[28]: import matplotlib.cm as cm

img = keras.utils.load_img(img_path)
img = keras.utils.img_to_array(img)

# Convert heatmap to RGB using the "jet" colormap
heatmap = np.uint8(255 * heatmap)
jet = cm.get_cmap("jet")
jet_colors = jet(np.arange(256))[:, :3]
jet_heatmap = jet_colors[heatmap]

jet_heatmap = keras.utils.array_to_img(jet_heatmap)
jet_heatmap = jet_heatmap.resize((img.shape[1], img.shape[0]))
jet_heatmap = keras.utils.img_to_array(jet_heatmap)

# Combine with original image
superimposed_img = jet_heatmap * 0.4 + img
superimposed_img = keras.utils.array_to_img(superimposed_img)

# Display the superimposed image
plt.figure(figsize=(6, 6))
plt.imshow(superimposed_img)
plt.axis("off")
```

```
plt.title("Class Activation Map (CAM)")  
plt.show()
```

Class Activation Map (CAM)

